NOTE

SIMD	Single Instruction - Multiple Data	Đây là một phân loại trong kiến trúc máy tính dùng để chỉ các bộ xử lý có khả năng thực hiện **cùng một lệnh** trên **nhiều đơn vị dữ liệu** cùng một lúc. Điều này giúp tăng tốc độ xử lý đáng kể đối với các tác vụ có tính lặp lại cao.
FPS	Frame per Second	
AST	Abstract Syntax Tree	Là một cấu trúc dữ liệu dạng cây dùng để đại diện cho cấu trúc logic của mã nguồn. Khi bạn viết mã (ví dụ: JavaScript), máy tính không hiểu ngay lập tức các dòng chữ đó. Nó cần chuyển đổi văn bản của bạn thành một "bản đồ" logic mà trình biên dịch (compiler) hoặc trình thông dịch (interpreter) có thể xử lý. Bản đồ đó chính là AST.
AOT	Ahead Of Time	**Biên dịch trước thời hạn** — là quá trình dịch mã nguồn của một ngôn ngữ lập trình sang mã máy (machine code) **trước khi** chương trình được thực hiện.
JIT	Just in Time	Thay vì dịch toàn bộ chương trình trước khi chạy (AOT) hoặc dịch từng dòng một cách chậm chạp khi chạy (Interpretation), JIT sẽ theo dõi chương trình và chỉ dịch những phần mã quan trọng sang mã máy ngay khi chúng chuẩn bị thực thi.

SharedArrayBuffer là một đối tượng trong JavaScript (thuộc chuẩn ECMAScript) cho phép bạn chia sẻ một vùng nhớ dữ liệu thô (raw binary data) giữa luồng chính (Main Thread) và các luồng phụ (Web Workers).

Garbage Collection (GC) — là một cơ chế **quản lý bộ nhớ tự động** trong lập trình máy tính. Nhiệm vụ chính của nó là tìm kiếm và giải phóng những vùng nhớ (object, biến, dữ liệu) mà chương trình không còn sử dụng nữa để nhường chỗ cho các dữ liệu mới.

- Quản lý tự động: Python, Java, JS
- Quản lý thủ công: C/C++

CHƯƠNG 1: GIỚI THIỆU

1.1. Lý do chọn đề tài

1.1.1. Sự dịch chuyển của các ứng dụng hiệu năng cao lên nền tảng Web

Trong những năm gần đây, trình duyệt web không còn chỉ là công cụ hiển thị văn bản và hình ảnh đơn giản mà đã trở thành một nền tảng thực thi các ứng dụng phức tạp như: chỉnh sửa video trực tuyến, trò chơi 3D, hội nghị truyền hình (WebRTC) và các hệ thống giám sát thông minh. Xu hướng "Web-first" đòi hỏi trình duyệt phải xử lý khối lượng dữ liệu khổng lồ mà trước đây chỉ có các ứng dụng Desktop (Native App) mới đảm đương nổi.

1.1.2. Thách thức về tính toán đối với nội dung video độ phân giải 4K

Sự phổ biến của nội dung 4K (3840x2160 pixel) đặt ra một bài toán khắc nghiệt về hiệu suất. Một khung hình 4K chứa khoảng **8,3 triệu pixel**. Để xử lý video mượt mà ở tốc độ 30 hoặc 60 FPS, hệ thống phải thực hiện hàng trăm triệu phép tính mỗi giây trên bộ nhớ đệm.

Đối với các thuật toán như **Motion Detection** (Phát hiện chuyển động), việc truy xuất và so sánh từng pixel giữa các khung hình liên tiếp tạo ra áp lực cực lớn lên CPU và băng thông bộ nhớ. Nếu hiệu năng không đủ tốt, hiện tượng trễ (latency), tụt khung hình (dropped frames) và quá tải tài nguyên hệ thống là điều khó tránh khỏi.

1.1.3. Giới hạn của JavaScript và sự trỗi dậy của WebAssembly

Mặc dù JavaScript đã có những bước tiến khổng lồ nhờ các bộ máy tối ưu hóa như V8 (Google) với cơ chế JIT compilation, nhưng bản chất là một ngôn ngữ có kiểu dữ liệu động (dynamically typed) và quản lý bộ nhớ tự động (Garbage Collection) khiến JS vẫn gặp khó khăn trong việc duy trì hiệu suất ổn định và dự đoán được cho các tác vụ tính toán nặng.

WebAssembly (Wasm) ra đời như một bước ngoặt, cho phép thực thi mã nhị phân với tốc độ gần như native code ngay trên trình duyệt. Wasm mang lại khả năng tận dụng tối đa sức mạnh của các ngôn ngữ bậc thấp như C/C++ hay Rust, giúp tối ưu hóa bộ nhớ và các lệnh tính toán song song (SIMD).

1.1.4. Ý nghĩa thực tiễn của đề tài

Việc lựa chọn đề tài "**So sánh hiệu năng giữa JavaScript và WebAssembly trong bài toán Motion Detection trên video 4K**" xuất phát từ nhu cầu thực tế:

- **Về mặt kỹ thuật:** Xác định rõ ngưỡng giới hạn của JS và điểm bắt đầu ưu thế của Wasm trong xử lý ảnh thời gian thực.

- **Về mặt ứng dụng:** Cung cấp cơ sở khoa học cho các nhà phát triển trong việc lựa chọn công nghệ tối ưu khi xây dựng các hệ thống giám sát, AI Camera hoặc các ứng dụng xử lý video trực tiếp trên trình duyệt web hiện đại.
- **Về mặt nghiên cứu:** Tìm hiểu sâu về cách thức vận hành của trình duyệt, cơ chế quản lý bộ nhớ và các phương pháp tối ưu hóa phần mềm cấp thấp.

NOTE:

- 4K nặng gấp 4 lần Full HD (1080p), điều này khiến JS bắt đầu "hụt hơi".
- **Tính ổn định:** Không chỉ nói về "nhanh" mà còn về "sự ổn định". JS có thể nhanh lúc đầu nhưng hay bị khựng (do JIT tối ưu lại hoặc Garbage Collector chạy), còn Wasm thì chạy "phẳng" một đường.

1.2. Mục tiêu nghiên cứu

Mục tiêu tổng quát của đề tài là nghiên cứu, triển khai và so sánh thực chứng hiệu năng giữa JavaScript và WebAssembly trong các tác vụ xử lý video độ phân giải 4K trên trình duyệt, từ đó đưa ra những đánh giá khách quan về khả năng ứng dụng của từng công nghệ.

Để đạt được mục tiêu tổng quát, đề tài tập trung vào các mục tiêu cụ thể sau:

- **Xây dựng ứng dụng thực nghiệm:** Thiết kế và cài đặt một hệ thống phát hiện chuyển động **(Motion Detection)** chạy trực tiếp trên trình duyệt, hỗ trợ hai chế độ thực thi là JavaScript thuần và WebAssembly (được biên dịch từ C++).
- **Thiết lập môi trường đo lường:** Xây dựng bộ công cụ theo dõi tự động các chỉ số hiệu năng hệ thống theo thời gian thực bao gồm:
    - **Tốc độ xử lý (FPS):** Số lượng khung hình xử lý được trong một giây.
    - **Thời gian trễ (Latency):** Thời gian tính toán chi tiết cho từng khung hình đơn lẻ (ms/frame).
    - **Tài nguyên hệ thống:** Mức độ chiếm dụng CPU và RAM.
- **Đánh giá sự ảnh hưởng của độ phân giải:** Thực hiện thử nghiệm trên các mức độ phân giải 4K - thời điểm mà WebAssembly thể hiện ưu thế vượt trội rõ ràng so với JavaScript.
- **Phân tích tính ổn định của hệ thống:** Nghiên cứu tác động của cơ chế JIT và Garbage Collection trong JavaScript đối với sự ổn định của khung hình, sau đó so sánh với hiệu suất của WebAssembly.

- **Đề xuất hướng tối ưu hóa:** Dựa trên kết quả thực nghiệm, đưa ra các khuyến nghị kỹ thuật về việc lựa chọn công nghệ và phương pháp tối ưu hóa (như quản lý bộ nhớ, truyền dữ liệu) cho các ứng dụng web xử lý hình ảnh nặng trong tương lai.

NOTE:

- **Tính khả thi:** Các mục tiêu trên cho thấy kế hoạch đo lường rất rõ ràng
- **Từ ngữ chuyên môn:** Sử dụng các cụm từ như *"ngưỡng phá vỡ"*, *"tính nhất quán"*, *"thực chứng"* sẽ làm tăng giá trị học thuật cho báo cáo của bạn.

1.3. Đối tượng và phạm vi nghiên cứu

1.3.1. Đối tượng nghiên cứu

- **Các nền tảng thực thi mã nguồn trên Web:**
    - **JavaScript:** Tập trung vào hiệu năng thực thi trên bộ máy V8 (Google Chrome).
    - **WebAssembly:** Mã nhị phân được biên dịch từ ngôn ngữ nguồn C++ thông qua bộ công cụ Emscripten.
- **Thuật toán xử lý ảnh:** Thuật toán phát hiện chuyển động dựa trên phương pháp trừ khung hình (Frame Differencing) và ngưỡng động (Thresholding).
- **Chỉ số đo lường hiệu năng:**
    - FPS (Frames Per Second)
    - Thời gian xử lý khung hình (Latency)
    - Mức độ tiêu thụ CPU và quản lý bộ nhớ RAM.

1.3.2. Phạm vi nghiên cứu

- **Môi trường thực hiện:**
    - Thực nghiệm trên các trình duyệt phổ biến hiện nay (ưu tiên Google Chrome).
    - Sử dụng máy tính cá nhân cấu hình phổ thông để đảm bảo tính khách quan của kết quả benchmark.
- **Dữ liệu đầu vào:**
    - Luồng video trực tiếp từ tệp video mẫu.
    - Các mức độ phân giải thử nghiệm: 4K - 3840x2160.

- **Giới hạn kỹ thuật:**
    - Tập trung vào so sánh hiệu năng thực thi trên CPU giữa JS và Wasm đơn luồng.
    - Các kỹ thuật nâng cao như WebGPU, Wasm đa luồng (Multi-threading) hoặc SIMD sẽ được đề cập ở phần hướng phát triển hoặc dùng làm đối chứng bổ trợ, không phải là nội dung cốt lõi của bài toán so sánh chính.
    - Không đi sâu vào độ chính xác của các mô hình học máy (AI/Deep Learning) phức tạp mà chỉ tập trung vào tốc độ xử lý pixel thuần túy của thuật toán Motion Detection cơ bản.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Cơ chế thực thi của JavaScript và Engine V8

2.1.1. Tổng quan về Engine V8

V8 là một dự án mã nguồn mở của Google được viết bằng C++, đóng vai trò là bộ máy thực thi JavaScript và WebAssembly cho trình duyệt Google Chrome. Khác với các ngôn ngữ biên dịch truyền thống (AOT - Ahead Of Time), V8 sử dụng mô hình biên dịch Just-In-Time (JIT) để chuyển đổi mã nguồn JavaScript trực tiếp sang mã máy ngay tại thời điểm thực thi.

2.1.2. Quy trình thực thi mã nguồn

Quá trình xử lý một đoạn mã JavaScript trong V8 diễn ra qua các giai đoạn chính sau:

- **Phân tích:** Trình phân tích cú pháp (Parser) nhận mã nguồn và chuyển đổi thành Cây cú pháp trừu tượng (Abstract Syntax Tree - AST).
- **Thông dịch với Ignition:** AST được đưa vào bộ thông dịch gọi là Ignition. Ignition chuyển đổi AST thành các dòng Bytecode. Đây là mã trung gian, giúp ứng dụng bắt đầu chạy ngay lập tức mà không cần chờ biên dịch toàn bộ.
- **Biên dịch tối ưu với TurboFan:** Trong khi Bytecode đang chạy, một bộ phận gọi là Profiler sẽ theo dõi các đoạn mã được thực thi lặp lại nhiều lần (gọi là "Hot Code"). Các đoạn mã này sẽ được gửi đến trình biên dịch tối ưu TurboFan. TurboFan dựa trên các thông tin thu thập được (về kiểu dữ liệu, cấu trúc đối tượng) để dịch Bytecode thành mã máy cực kỳ tối ưu.

2.1.3. Cơ chế JIT và Tối ưu hóa dựa trên giả định (Speculative Optimization)

Điểm đặc biệt của V8 là khả năng "đoán" kiểu dữ liệu. Vì JavaScript là ngôn ngữ có kiểu động (Dynamic Typing), V8 giả định rằng nếu một hàm luôn nhận vào số nguyên (Integer) trong 100 lần trước đó, thì lần thứ 101 cũng sẽ là số nguyên. Nhờ giả định này, TurboFan có thể tạo ra mã máy rất gọn nhẹ, bỏ qua các bước kiểm tra kiểu phức tạp.

2.1.4. Cơ chế Hủy tối ưu (Deoptimization)

Đây là "điểm yếu" về mặt hiệu năng của JavaScript trong các tác vụ nặng:

- Nếu giả định của V8 sai (ví dụ: hàm đang xử lý số nguyên bỗng nhiên nhận vào một chuỗi hoặc số thực), hệ thống sẽ gặp lỗi thực thi ở mức mã máy.
- Ngay lập tức, V8 phải thực hiện quá trình Deoptimization: Vứt bỏ bản mã máy đã tối ưu và quay lại chạy bằng Bytecode thông qua bộ thông dịch Ignition.

- Quá trình này gây ra hiện tượng "khựng" (Jitter) về FPS, ảnh hưởng trực tiếp đến sự ổn định khi xử lý video 4K thời gian thực.

2.1.5. Quản lý bộ nhớ và Garbage Collection (GC)

V8 quản lý bộ nhớ tự động thông qua cơ chế dọn rác (Garbage Collection). Khi xử lý video 4K, lượng dữ liệu pixel được tạo ra và thay thế liên tục trong bộ nhớ Heap.

- Orinoco: Là dự án dọn rác hiện đại của V8, giúp thực hiện dọn dẹp song song (Parallel) và tăng tiến (Incremental).
- Tuy nhiên, khi bộ nhớ đầy, các "GC Pause" (tạm dừng để dọn rác) vẫn có thể xảy ra, làm gián đoạn luồng xử lý hình ảnh và gây tụt khung hình.

2.2. WebAssembly (Wasm)

2.2.1. Khái niệm và Mục tiêu ra đời

WebAssembly (viết tắt là Wasm) không phải là một ngôn ngữ lập trình theo nghĩa thông thường, mà là một **định dạng chỉ dẫn nhị phân (binary instruction format)** cho một máy ảo dựa trên ngăn xếp (stack-based virtual machine).

Wasm được thiết kế như một mục tiêu biên dịch cho các ngôn ngữ bậc thấp như C, C++, và Rust, cho phép triển khai các ứng dụng có hiệu năng cực cao trên trình duyệt web với tốc độ thực thi gần như tương đương với phần mềm chạy trực tiếp trên hệ điều hành (near-native speed).

2.2.2. Kiến trúc Máy ảo Ngăn xếp (Stack-based Machine)

Khác với kiến trúc thanh ghi (register-based) của nhiều CPU hiện đại, Wasm sử dụng mô hình ngăn xếp để thực hiện các phép toán.

- Các chỉ dẫn (instructions) đẩy dữ liệu vào ngăn xếp và lấy dữ liệu ra để xử lý.
- Ưu điểm của mô hình này là mã nhị phân rất gọn nhẹ, dễ dàng xác thực (validation) và biên dịch nhanh chóng sang mã máy của bất kỳ kiến trúc CPU nào (x86, ARM).

2.2.3. Định dạng nhị phân và Hiệu năng nạp (Loading Performance)

JavaScript là mã nguồn dạng văn bản (textual source), trình duyệt phải tải về, phân tích cú pháp (parse) và biên dịch. Ngược lại, Wasm là định dạng nhị phân đã được tối ưu sẵn:

- Kích thước nhỏ: Giúp giảm thời gian tải qua mạng.

- Biên dịch nhanh: Vì đã ở dạng gần với mã máy, trình duyệt có thể biên dịch Wasm sang mã native với tốc độ cực nhanh, thậm chí biên dịch song song khi đang tải dữ liệu (streaming compilation).

2.2.4. Bộ nhớ tuyến tính (Linear Memory)

Đây là đặc điểm quan trọng nhất đối với bài toán xử lý video 4K của bạn:

- Wasm sử dụng một mảng các byte liên tục, có thể mở rộng, gọi là Linear Memory.
- Dữ liệu hình ảnh (pixel) từ Canvas được đẩy trực tiếp vào vùng nhớ này dưới dạng mảng thô (ArrayBuffer).
- Wasm có quyền truy cập trực tiếp vào vùng nhớ này với độ trễ cực thấp, không bị ảnh hưởng bởi các lớp trừu tượng hay cơ chế quản lý đối tượng phức tạp của JavaScript.
- Tính toán: Với khung hình 4K, dung lượng bộ nhớ cần thiết là:
  3840 x 2160 x 4 bytes (RGBA) ~ 33.18 MB/frame
  Wasm quản lý vùng nhớ này một cách thủ công (giống như C++), giúp kiểm soát chính xác tài nguyên và tránh việc cấp phát/giải phóng bộ nhớ liên tục.

2.2.5. Quy trình biên dịch từ C++ sang Wasm (Emscripten)

Để chạy được thuật toán Motion Detection bằng Wasm, mã nguồn C++ đi qua chuỗi công cụ (toolchain) Emscripten:

1. Frontend (Clang): Biên dịch C++ sang mã trung gian LLVM IR.
2. Optimizer: Tối ưu hóa mã nguồn ở mức độ hệ thống.
3. Backend: Chuyển đổi LLVM IR thành tệp .wasm và tạo ra một tệp "glue code" bằng JavaScript để làm cầu nối liên lạc giữa trình duyệt và Wasm.

2.2.6. Ưu điểm so với JavaScript JIT

Khác với sự biến động của JIT (như đã trình bày ở mục 2.1), hiệu năng của WebAssembly có tính dự đoán cao (predictable performance):

- Không có quá trình "Warm-up" (làm nóng). Wasm đạt tốc độ tối đa ngay từ khung hình đầu tiên.
- Không có hiện tượng Deoptimization: Vì kiểu dữ liệu đã được xác định cứng từ lúc biên dịch, không có giả định sai lệch nào xảy ra.
- Không bị gián đoạn bởi Garbage Collection: Wasm tự quản lý bộ nhớ, giúp FPS duy trì một đường thẳng tắp, cực kỳ quan trọng cho xử lý video thời gian thực.

2.3. Xử lý video và hình ảnh trên Web

2.3.1. Cơ chế thu nhận luồng Video

Trên trình duyệt, dữ liệu video thường được nạp qua thẻ <video> từ hai nguồn chính: tệp tin đa phương tiện (MP4, WebM) hoặc luồng trực tiếp từ thiết bị (Webcam) thông qua API getUserMedia. Tuy nhiên, thẻ <video> chỉ đóng vai trò hiển thị (display-only). Để can thiệp vào từng pixel nhằm phát hiện chuyển động, dữ liệu phải được chuyển sang một "vùng đệm đồ họa" có thể truy cập được, đó chính là Canvas API.

2.3.2. Canvas API và Image Extraction

Canvas đóng vai trò là "trạm trung chuyển" dữ liệu. Quy trình trích xuất khung hình diễn ra như sau:

1. Sử dụng phương thức drawImage() để vẽ khung hình hiện tại của video lên một đối tượng Canvas ngầm (Offscreen Canvas).
2. Sử dụng phương thức getImageData() để trích xuất mảng pixel từ Canvas vào bộ nhớ RAM dưới dạng đối tượng ImageData.

2.3.3. Cấu trúc dữ liệu ImageData và Uint8ClampedArray

Dữ liệu hình ảnh trong trình duyệt được lưu trữ theo định dạng RGBA (Red, Green, Blue, Alpha).

- Mỗi pixel được đại diện bởi 4 byte (mỗi kênh màu 1 byte, giá trị từ 0 đến 255).
- Toàn bộ khung hình được lưu trong một mảng một chiều gọi là Uint8ClampedArray.

Công thức tính chỉ số (Index) của pixel:

Để truy cập vào giá trị màu của một pixel tại tọa độ (x, y), ta sử dụng công thức:

Index = (y x Width + x) x 4

- Index: Màu đỏ (Red)
- Index + 1: Màu xanh lá (Green)
- Index + 2: Màu xanh dương (Blue)
- Index + 3: Độ trong suốt (Alpha)

2.3.4. Thách thức về tài nguyên với độ phân giải 4K

Khi xử lý video 4K (3840 x 2160), khối lượng dữ liệu mà trình duyệt phải xử lý trong mỗi khung hình là cực kỳ lớn:

- Tổng số pixel: 3840 x 2160 = 8.294.400 pixel.
- Kích thước bộ nhớ: 8.294 .400 x 4 bytes ~ 33,18 MB cho mỗi khung hình.
- Băng thông tính toán: Để đạt tốc độ 60 FPS, hệ thống chỉ có khoảng 16,6ms để hoàn thành việc trích xuất, truyền tải và thực hiện thuật toán so sánh trên 33MB dữ liệu này.

2.3.5. Sự tương thích giữa JS Typed Arrays và Wasm Memory

Điểm mấu chốt giúp WebAssembly xử lý ảnh hiệu quả là nhờ vào Typed Arrays (Uint8ClampedArray).

- Trong JavaScript, các mảng thông thường (Array) là các đối tượng phức tạp. Nhưng Typed Arrays lại là những vùng nhớ thô liên tục (Raw binary data).
- Cấu trúc này tương đồng hoàn toàn với cách quản lý bộ nhớ trong C++ và Linear Memory của Wasm. Do đó, ta có thể "ánh xạ" (map) vùng nhớ pixel từ JS sang Wasm một cách nhanh chóng mà không cần chuyển đổi định dạng dữ liệu phức tạp.

2.4. Thuật toán Phát hiện chuyển động (Motion Detection)

2.4.1. Nguyên lý cơ bản: Trừ khung hình (Frame Differencing)

Phương pháp phổ biến và hiệu quả nhất để thực hiện trên trình duyệt là Frame Differencing. Nguyên lý của nó dựa trên việc so sánh sự thay đổi giá trị cường độ sáng của từng pixel tại cùng một tọa độ giữa hai khung hình liên tiếp trong luồng video: khung hình hiện tại (Ft) và khung hình trước đó (F_t-1).

Nếu sự khác biệt vượt quá một ngưỡng nhất định, pixel đó được coi là đã có sự chuyển động.

2.4.2. Công thức toán học

Để tối ưu hóa tốc độ, thay vì so sánh cả 3 kênh màu (RGB), thuật toán thường chuyển đổi pixel sang dạng mức xám (Grayscale) hoặc tính toán tổng độ lệch tuyệt đối.

Công thức tính độ lệch tuyệt đối cho một pixel tại tọa độ (x, y):

$$\Delta P(x, y) = |R_t - R_{t-1}| + |G_t - G_{t-1}| + |B_t - B_{t-1}|$$

Sau đó, giá trị ΔP được so sánh với một giá trị ngưỡng ($Threshold$):

- Nếu $\Delta P(x, y) > Threshold$: Pixel có chuyển động.
- Nếu $\Delta P(x, y) \leq Threshold$: Pixel tĩnh.

2.4.3. Quy trình thực hiện thuật toán (Pipeline)

Thuật toán được triển khai qua các bước tuần tự sau:

1. Lưu trữ khung hình cũ: Lưu mảng pixel của khung hình $t-1$ vào một vùng đệm (Buffer).
2. Đọc khung hình mới: Lấy mảng pixel của khung hình t từ Canvas.
3. Vòng lặp quét pixel: Duyệt qua toàn bộ $8.294.400$ pixel (đối với 4K).
4. Tính toán sai biệt: Thực hiện phép trừ và lấy giá trị tuyệt đối cho từng kênh màu.
5. Ghi đè kết quả: * Để hiển thị trực quan, các pixel có chuyển động thường được tô màu đỏ hoặc trắng.
    - Các pixel không chuyển động được giữ nguyên hoặc làm mờ.
6. Cập nhật: Gán khung hình hiện tại thành khung hình cũ để chuẩn bị cho chu kỳ tiếp theo.

2.4.4. Thách thức về tính toán trong bài toán 4K

Đối với video 4K, thuật toán đơn giản này trở nên cực kỳ "đắt đỏ" về mặt tài nguyên:

- Số phép tính: Với mỗi khung hình, hệ thống phải thực hiện ít nhất:
    - 8,3 triệu x 3 phép trừ.
    - 8,3 triệu x 3 phép lấy giá trị tuyệt đối.
    - 8,3 triệu phép so sánh điều kiện.
- Tổng cộng: Hơn 50 triệu phép toán cần hoàn thành trong vòng 16ms để đạt 60 FPS.

2.4.5. Tại sao thuật toán này phù hợp để Benchmark?

Thuật toán Motion Detection bằng Frame Differencing là đối tượng hoàn hảo để so sánh JS và Wasm vì:

- Tính lặp lại cao: Vòng lặp for chạy qua 8 triệu phần tử là kịch bản lý tưởng để JIT của JS thể hiện khả năng tối ưu hóa, nhưng cũng là nơi Wasm thể hiện sự ổn định của mã máy.
- Truy cập bộ nhớ liên tục: Kiểm tra khả năng đọc/ghi vào *ArrayBuffer* và *Linear Memory* của cả hai nền tảng.
- Độ phức tạp O(n): Giúp kết quả đo lường FPS phản ánh trực tiếp năng lực tính toán của Engine mà không bị nhiễu bởi các logic nghiệp vụ phức tạp khác.

CHƯƠNG 3: THIẾT KẾ VÀ CÀI ĐẶT HỆ THỐNG

3.1. Kiến trúc hệ thống

Để thực hiện so sánh hiệu năng một cách khách quan, hệ thống được thiết kế theo mô hình kiến trúc phân lớp, tách biệt giữa luồng xử lý giao diện (UI Thread) và luồng tính toán (Worker Thread). Điều này giúp việc đo lường các chỉ số như FPS và thời gian xử lý không bị ảnh hưởng bởi các tác vụ vẽ đồ họa của trình duyệt.

3.1.1. Sơ đồ khối tổng quát

Hệ thống bao gồm 3 thành phần chính:

1. **Lớp Dữ liệu đầu vào (Input Layer):** Tiếp nhận luồng video 4K từ thẻ <video> và trích xuất dữ liệu pixel thông qua Offscreen Canvas.
2. **Lớp Thực thi (Execution Layer):** Đây là trọng tâm của đồ án, bao gồm hai module độc lập:
    - Module JavaScript: Thực thi thuật toán trực tiếp trên môi trường V8.
    - Module WebAssembly: Thực thi mã nhị phân được biên dịch từ C++, tương tác với bộ nhớ tuyến tính (Linear Memory).
3. **Lớp Giám sát và Hiển thị (Monitoring & UI Layer):** Sử dụng Performance API để thu thập số liệu và hiển thị kết quả xử lý ra màn hình người dùng.

3.1.2. Luồng xử lý dữ liệu (Data Flow)

Quy trình xử lý một khung hình 4K diễn ra theo các bước tuần tự để đảm bảo tính nhất quán:

1. **Capture**: Trích xuất khung hình hiện tại thành mảng Uint8ClampedArray.
2. **Transfer**: Dữ liệu được gửi tới Worker (để tránh làm treo trình duyệt).
    - Đối với JS: Dữ liệu được truyền dưới dạng tham chiếu hoặc sao chép vào mảng nội bộ.
    - Đối với Wasm: Dữ liệu được sao chép vào vùng nhớ Linear Memory mà Wasm có thể truy cập trực tiếp bằng con trỏ (pointers).
3. **Process**: Thực hiện thuật toán Motion Detection (đã mô tả ở mục 2.4).
4. **Feedback**: Trả về kết quả (mảng pixel đã xử lý) và các thông số thời gian (tính bằng micro giây).

3.1.3. Cơ chế giao tiếp giữa JavaScript và WebAssembly

Vì WebAssembly không thể truy cập trực tiếp vào DOM hoặc các đối tượng JavaScript phức tạp, hệ thống sử dụng một vùng nhớ dùng chung gọi là Shared Memory Buffer.

- JavaScript sẽ đóng vai trò "người điều phối": nạp dữ liệu pixel vào vùng nhớ này.
- WebAssembly sẽ thực hiện tính toán "tại chỗ" (in-place) trên vùng nhớ đó để tránh việc sao chép dữ liệu dư thừa, vốn là tác vụ rất tốn kém đối với độ phân giải 4K (33MB/khung hình).

3.1.4. Thiết kế module đo lường (Benchmarking Module)

Hệ thống tích hợp một bộ đo độc lập để ghi lại các chỉ số:

- **Time-to-Process:** Sử dụng `performance.now()` để đo thời gian từ lúc bắt đầu thuật toán đến lúc kết thúc.
- **Memory Footprint:** Theo dõi qua `performance.memory` (trên Chrome) để đánh giá sự biến động của bộ nhớ Heap.
- **FPS Counter:** Tính toán dựa trên số khung hình thực tế xử lý được trong khoảng thời gian 1000ms.

3.2. Cài đặt thực nghiệm

3.2.1. Môi trường thử nghiệm

Để kết quả benchmark có giá trị tham chiếu, các thử nghiệm được tiến hành trên một hệ thống thống nhất với cấu hình như sau:

- **Phần cứng (Hardware):**
  - **CPU:** Apple M4 – Quyết định tốc độ xử lý tính toán.
  - **RAM:** 48GB – Ảnh hưởng đến khả năng lưu trữ các buffer 4K.
  - **GPU:** [] – Hỗ trợ render kết quả ra Canvas.
- **Phần mềm (Software):**
  - **Hệ điều hành:** MacOS Sequoia.
  - **Trình duyệt:** Google Chrome – Sử dụng Engine V8 làm nền tảng so sánh.
  - **Trình biên dịch Wasm:** Emscripten (emcc).

3.2.2. Triển khai thuật toán bằng JavaScript

Trong bản thực nghiệm này, thuật toán Phát hiện chuyển động được triển khai bằng JavaScript thuần (Vanilla JS), tập trung vào việc tối ưu hóa cấu trúc vòng lặp và quản lý bộ nhớ đệm để đảm bảo tính công bằng khi so sánh với WebAssembly.

Quy trình xử lý được chia thành 3 giai đoạn chính:

A. Tiền xử lý: Chuyển đổi mức xám (Grayscale Conversion)

Thay vì xử lý trực tiếp trên dữ liệu RGBA (4 kênh màu), hệ thống thực hiện chuyển đổi ảnh về dạng mức xám (1 kênh duy nhất). Việc này giúp giảm lượng dữ liệu cần xử lý ở các bước sau đi 4 lần.

- **Kỹ thuật:** Sử dụng công thức trọng số chuẩn ITU-R BT.601:
  Gray = 0.299R + 0.587G + 0.114B
- **Tối ưu:** Dữ liệu được lưu trữ trong Uint8ClampedArray, một dạng **TypedArray** giúp Engine V8 cấp phát bộ nhớ liên tục và tối ưu hóa tốc độ truy cập.

B. Khử nhiễu: Thuật toán Box Blur (Spatial Filtering)

Đây là giai đoạn tiêu tốn tài nguyên nhất trong mã nguồn JavaScript. Một bộ lọc làm mờ (Blur) được áp dụng để loại bỏ các điểm nhiễu (noise) nhỏ từ cảm biến camera, tránh việc phát hiện chuyển động giả.

- **Cơ chế:** Sử dụng ma trận tích chập (Convolution Kernel) với bán kính r=3 (tương đương ma trận 7x7 pixel).
- **Độ phức tạp:** Với mỗi pixel, JS phải thực hiện cộng dồn 49 giá trị lân cận. Ở độ phân giải 4K, số phép tính cộng phát sinh xấp xỉ:
  8,3 triệu pixel x 49 phép tính ~ 406 triệu phép tính/khung hình
- **Thách thức cho JIT:** Các vòng lặp lồng nhau (nested loops) ở cường độ cao này sẽ kích hoạt trình biên dịch tối ưu **TurboFan** của V8 để cố gắng chuyển đổi mã nguồn sang mã máy.

C. Phân tích chuyển động: Frame Differencing & Thresholding

Sau khi làm mờ, khung hình hiện tại được so sánh với khung hình trước đó (prevFrameGray) được lưu trữ trong bộ nhớ đệm.

- **Xử lý logic:** Sử dụng Math.abs() để tính độ lệch tuyệt đối giữa hai giá trị pixel xám.
- **Ngưỡng động (Thresholding):** Nếu $\Delta > 30$, pixel kết quả sẽ được đặt là 255 (Trắng - Có chuyển động), ngược lại là 0 (Đen - Tĩnh).
- **Quản lý trạng thái:** Biến prevFrameGray được duy trì bên ngoài hàm xử lý để làm tham chiếu cho khung hình kế tiếp, mô phỏng cơ chế xử lý luồng (streaming).

D. Các kỹ thuật tối ưu hóa mã nguồn đã áp dụng

Để bản JS đạt được hiệu năng cao nhất có thể, các kỹ thuật sau đã được triển khai:

1. **Tránh "Memory Leak" và "Garbage Collection":** Sử dụng các mảng đệm cố định, hạn chế việc khởi tạo lại đối tượng bên trong vòng lặp chính.

2. **Sử dụng truy cập mảng một chiều:** Chuyển đổi tọa độ 2D (x, y) thành chỉ số 1D để tối ưu hóa việc quét bộ nhớ tuần tự (Sequential Memory Access).
3. **Hoạt động đơn luồng:** Toàn bộ thuật toán chạy trên một luồng duy nhất để làm cơ sở đối chứng với bản WebAssembly đơn luồng.

3.2.3. Triển khai thuật toán bằng C++ và WebAssembly (Wasm)

Bản thực hiện này sử dụng ngôn ngữ C++ làm ngôn ngữ nguồn, sau đó biên dịch sang định dạng nhị phân .wasm thông qua bộ công cụ Emscripten. Mục tiêu là tận dụng tối đa khả năng tính toán của CPU thông qua mã máy đã được tối ưu hóa trước (Ahead-Of-Time).

A. Quản lý bộ nhớ và Cơ chế Buffer

Khác với JavaScript, phiên bản C++ kiểm soát chặt chẽ việc cấp phát bộ nhớ để tránh overhead lúc thực thi:

- **Linear Memory:** Sử dụng các đối tượng std::vector để giữ dữ liệu bên trong vùng nhớ tuyến tính của Wasm.
- **Hàm ensure_buffers:** Đây là một kỹ thuật tối ưu quan trọng. Thay vì giải phóng và cấp phát lại bộ nhớ cho mỗi khung hình 4K (vốn cực kỳ tốn kém), hệ thống chỉ thực hiện resize khi kích thước video thay đổi. Điều này giúp duy trì một vùng nhớ ổn định, giảm thiểu sự can thiệp của bộ dọn rác (mặc dù Wasm không có GC, nhưng việc cấp phát lại ở mức hệ thống vẫn gây trễ).
- **Giao tiếp con trỏ:** Hàm processMotion nhận vào một con trỏ const uint8_t* rgba và trả về một con trỏ uint8_t*. Việc truyền địa chỉ vùng nhớ thay vì sao chép toàn bộ mảng dữ liệu giúp giảm thiểu độ trễ truyền dẫn (Data Transfer Latency).

B. Quy trình xử lý song song mức thấp

Mặc dù logic thuật toán tương đồng với bản JavaScript, nhưng cách C++ thực thi mang lại hiệu quả vượt trội:

1. **Chuyển đổi Grayscale:** Sử dụng kiểu dữ liệu tĩnh (uint8_t) giúp trình biên dịch tối ưu hóa các phép tính số thực sang các tập lệnh CPU chuyên dụng.
2. **Box Blur (Ma trận tích chập):** * Với r=3, vòng lặp 4 tầng (4-level nested loops) được C++ xử lý hiệu quả nhờ khả năng **Sắp xếp thanh ghi (Register Allocation)** và **Dự đoán nhánh (Branch Prediction)** của trình biên dịch.
    - Khi biên dịch với cờ -O3, các vòng lặp này có thể được trình biên dịch "trải phẳng" (loop unrolling) để tăng tốc độ truy xuất.
3. **Frame Differencing:** Phép toán so sánh độ lệch tuyệt đối std::abs được thực hiện trực tiếp trên các giá trị số nguyên trong bộ nhớ, loại bỏ hoàn toàn các bước kiểm tra kiểu dữ liệu (type checking) vốn là rào cản của JS.

C. Các hàm bổ trợ và Export

Sử dụng macro EMSCRIPTEN_KEEPALIVE và khối extern "C" để đảm bảo các hàm không bị lược bỏ (dead code elimination) và có thể gọi được từ môi trường JavaScript:

- **resetMotionDetector:** Giải phóng các vùng đệm, đưa hệ thống về trạng thái ban đầu.
- **getChangedPixelCount:** Trả về số lượng pixel có chuyển động—một chỉ số quan trọng để đánh giá mức độ hoạt động (Activity Level) của video mà không cần tốn thêm chi phí truyền mảng kết quả ngược lại JS nếu không cần hiển thị.

D. Tại sao phiên bản này lại mạnh mẽ hơn trong bài toán 4K?

1. **Tính ổn định (Deterministic):** Không có sự trồi sụt FPS do JIT biên dịch lại hay Garbage Collection dọn dẹp bộ nhớ.
2. **Pointer Arithmetic:** C++ truy cập bộ nhớ thông qua các phép toán con trỏ trực tiếp, giúp việc quét qua 8.3 triệu pixel diễn ra với tốc độ tối đa của phần cứng.
3. **Tối ưu hóa biên dịch:** Mã nguồn C++ được tối ưu hóa ở mức độ hệ thống bởi bộ tối ưu hóa LLVM trước khi đóng gói thành file .wasm.

3.2.4. Tối ưu hóa truyền dẫn dữ liệu (Memory Binding)

Điểm quan trọng nhất trong cài đặt thực nghiệm là cách chuyển 33MB dữ liệu 4K giữa JS và Wasm:

- Hệ thống khởi tạo một vùng nhớ Wasm Linear Memory cố định.
- JavaScript sử dụng phương thức `Module.HEAPU8.set()` để sao chép dữ liệu từ Canvas vào vùng nhớ của Wasm.
- Sau khi Wasm xử lý xong, JavaScript truy cập trực tiếp vào cùng vùng nhớ đó để lấy kết quả hiển thị, giúp giảm thiểu tối đa số lần sao chép dữ liệu không cần thiết.

3.2.5. Công cụ và phương pháp đo lường

- **Đo thời gian:** Sử dụng API performance.now() với độ chính xác đến micro giây (μs) để đo khoảng thời gian thực thi thuật toán (Execution Time).

- **Đo FPS:** Tính toán dựa trên số lượng khung hình xử lý thành công trong mỗi chu kỳ 1 giây.

- **Đo tài nguyên:** Sử dụng công cụ **Chrome DevTools Performance Monitor** để ghi lại mức độ chiếm dụng CPU (%) và dung lượng bộ nhớ Heap (MB) trong suốt quá trình chạy benchmark.

3.3. Kịch bản Benchmark

Để có cái nhìn toàn diện về năng lực tính toán của WebAssembly so với JavaScript trên video 4K, hệ thống được thiết lập để thực nghiệm qua 5 kịch bản với các mức độ tối ưu hóa tăng dần. Việc chia nhỏ các kịch bản này giúp cô lập và đánh giá chính xác giá trị của từng kỹ thuật phần cứng (SIMD, Multi-threading) mang lại.

3.3.1. Kịch bản 1: So sánh Baseline (JavaScript) với Wasm Scalar (C++)

- **Mô tả:**
  - Sử dụng mã nguồn JavaScript đã trình bày ở mục 3.2.2. Đây là mốc chuẩn đại diện cho hiệu năng của hầu hết các ứng dụng web hiện nay.
  - Mã nguồn C++ được biên dịch sang WebAssembly mà không bật các tính năng phần cứng đặc biệt.
- **Đặc điểm:** Chạy trên một luồng duy nhất (Main Thread/Worker), dựa hoàn toàn vào khả năng tối ưu hóa của JIT Compiler trong Engine V8.
- **Mục tiêu:** So sánh sự chênh lệch hiệu năng thuần túy giữa mã máy nhị phân (Wasm) và mã JIT (JavaScript) khi cả hai cùng chạy đơn luồng và xử lý dữ liệu tuần tự.

3.3.2. Kịch bản 2: Wasm SIMD (Single Instruction, Multiple Data)

- **Mô tả:** Bật cờ tối ưu hóa SIMD (`-msimd128`) khi biên dịch. Kỹ thuật này sử dụng các thanh ghi 128-bit của CPU để thực hiện một phép tính trên nhiều pixel cùng lúc.
- **Giá trị thực tiễn:** Trong thuật toán Motion Detection, SIMD cho phép xử lý đồng thời 4 hoặc 16 pixel trong một chu kỳ lệnh, rất phù hợp với các ma trận lọc (Box Blur) nặng nề của video 4K.

3.3.3. Kịch bản 3: Wasm Threads (Xử lý đa luồng)

- **Mô tả:** Sử dụng Web Workers kết hợp với `SharedArrayBuffer` để chia nhỏ khung hình 4K thành các phần (tiles/rows). Mỗi phần được xử lý bởi một nhân CPU độc lập thông qua Wasm.
- **Mục tiêu:** Đánh giá khả năng mở rộng (Scalability) của hệ thống khi tận dụng tối đa số nhân của CPU hiện đại.

3.3.4. Kịch bản 4: Wasm Ultimate (SIMD + Threads)

- **Mô tả:** Kết hợp cả hai kỹ thuật tối ưu hóa mạnh mẽ nhất: Vector hóa (SIMD) và Song song hóa (Threads).
- **Kỳ vọng:** Đây là kịch bản đại diện cho "đỉnh cao hiệu năng" trên nền tảng Web, tiệm cận nhất với tốc độ của ứng dụng Native cài đặt trực tiếp trên máy tính.

CHƯƠNG 4: KẾT QUẢ VÀ PHÂN TÍCH

4.1. So sánh Tốc độ xử lý (FPS & Processing Time)

- Biểu đồ FPS trung bình ở 720p, 1080p và 4K.
- Phân tích thời gian xử lý 1 frame (ms/frame). Tại sao ở 4K, JS thường vượt ngưỡng 16ms (không đạt 60 FPS)?

4.2. So sánh Tiêu thụ tài nguyên (CPU & RAM)

- Phân tích mức độ chiếm dụng CPU.
- Phân tích biểu đồ RAM (Memory Footprint). Nhận xét về tác động của Garbage Collection trong bản JS.

4.3. Phân tích độ ổn định (Jitter Analysis)

- So sánh độ lệch chuẩn (Standard Deviation) của thời gian xử lý giữa JS và Wasm.
- Hiện tượng "Warm-up": Hiệu năng JS thay đổi thế nào sau 10-20 giây đầu tiên nhờ JIT.

4.4. Đánh giá Overhead

- Chi phí thời gian khi chuyển dữ liệu từ Canvas sang bộ nhớ của Wasm.

CHƯƠNG 5: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Kết luận

- Tổng kết lại các con số chênh lệch (ví dụ: Wasm nhanh hơn JS bao nhiêu lần ở 4K).
- Xác định "điểm gãy" hiệu năng: Khi nào bắt buộc phải dùng Wasm?

5.2. Ưu và nhược điểm của từng phương pháp

- JS: Dễ phát triển, linh hoạt cho app nhỏ.
- Wasm: Phức tạp trong cài đặt nhưng hiệu suất ổn định cho app nặng.

5.3. Hướng phát triển (Future Work)

- Tối ưu hóa bằng SIMD trong Wasm để tăng tốc gấp đôi.
- Nghiên cứu tích hợp WebGPU để đưa toàn bộ việc xử lý 4K lên card đồ họa.
- Sử dụng Web Workers để xử lý đa luồng (Multi-threading).

TÀI LIỆU THAM KHẢO

Phụ lục

- Mã nguồn quan trọng của thuật toán (JS & C++).
- Bảng số liệu chi tiết các lần chạy Benchmark.
- Hình ảnh minh họa kết quả phát hiện chuyển động.

Một số lưu ý nhỏ để làm đồ án tốt hơn:

1. **Về hình ảnh:** Bạn nên chụp màn hình **Chrome Task Manager** khi đang chạy benchmark để làm bằng chứng thực tế cho CPU/RAM.
2. **Về biểu đồ:** Sử dụng biểu đồ đường (Line chart) cho FPS và biểu đồ cột (Bar chart) để so sánh thời gian xử lý giữa các độ phân giải.
3. **Về mã nguồn:** Hãy highlight các đoạn code xử lý vòng lặp pixel, vì đó là nơi Wasm thể hiện sức mạnh vượt trội nhất.